LAPORAN PRAKTIKUM

- **Identitas Praktikum**

  Nama MK : Praktikum Pemrograman Berorientasi Objek
  Kode MK : CAK2KAB4
  Bobot SKS : 3 SKS
  Tempat : LAB-MM
  Hari, tanggal : JUMAT, 10 APRIL 2026
  Jam : 12.30-SELESAI
  Topik praktikum : Modul-5

- **Identitas Mahasiswa**

  Nama lengkap : M.HANIF AL FAIZ
  NIM : 103112400042
  Program Studi : S-1 Teknik Informatika

- **Algoritma / studi kasus (jika ada) & Penjelasan**

  Study case:
  Sistem informasi akademik sederhana untuk universitas yang mencatat data Fakultas, Program Studi (Prodi), dan Dosen. Sistem ini harus mematuhi aturan relasi antar objek (Class) sebagai berikut:
  - Prodi berada di dalam Fakultas (Komposisi / Composition). Artinya, jika Fakultas dihapus, maka Prodi di dalamnya ikut terhapus karena Prodi diciptakan spesifik untuk Fakultas tersebut.
  - Dosen tetap ada walaupun tidak terdaftar di Prodi (Agregasi / Aggregation). Artinya objek Dosen bisa diarsipkan secara independen tanpa harus dihapus meski ia tidak mengajar di mana pun.
  - Dosen mengajar di Prodi (Asosiasi / Association). Ini menandakan adanya hubungan penggunaan/pengaitan antara Dosen dan Prodi.

  Algoritma:
  1. Mulai (Start).
  2. Buat objek-objek Dosen secara independen (contoh: Dosen Budi, Siti, Andi).
     -> Membuktikan relasi agregasi di mana dosen hidup mandiri.
  3. Buat objek Fakultas (contoh: Fakultas "Ilmu Komputer").
  4. Tambahkan Prodi ke dalam Fakultas tersebut (contoh: "Teknik Informatika" dan "Sistem Informasi").

-> Pada langkah ini, instansiasi (pembuatan) objek Prodi dilakukan di dalam metode Fakultas (Komposisi).

5. Ambil referensi objek Prodi yang baru dibuat dari dalam Fakultas.
6. Masukkan (daftarkan) objek Dosen ke dalam Prodi melalui metode tambahDosen(). Dosen bisa ditambahkan ke lebih dari 1 Prodi.
   -> Membuktikan relasi asosiasi antar objek.
7. Tampilkan seluruh struktur data (Fakultas, Prodi, dan Dosen).
8. Panggil metode mengajar() dari setiap objek Dosen untuk menunjukkan perilaku / perilaku mandirinya.
9. Selesai (End).

PENJELASAN RELASI PADA KODE:

A. Class Dosen
- Menyimpan atribut 'nidn' dan 'nama'.
- Memiliki metode utama 'mengajar()'.
- Class ini paling independen karena tidak menyimpan referensi Class lain di dalam dirinya.

B. Class Prodi
- Menyimpan atribut 'namaProdi' dan sebuah koleksi (ArrayList) bernama 'daftarDosen'.
- Relasi: Agregasi & Asosiasi terhadap Dosen.
- Penjelasan: Prodi menampung Dosen, namun metode constructor Prodi tidak menciptakan objek Dosen. Objek Dosen dilempar / dikirim dari luar melalui argumen pada metode `tambahDosen(Dosen dosen)`.

C. Class Fakultas
- Menyimpan atribut 'namaFakultas' dan sebuah koleksi (ArrayList) bernama 'daftarProdi'.
- Relasi: Komposisi terhadap Prodi.
- Penjelasan: Pada metode `tambahProdi(String namaProdi)`, Fakultas menggunakan argumen string untuk MENCIPTAKAN / MENGINSTANSIASI objek Prodi yang baru (`new Prodi(namaProdi)`). Jika institusi Fakultas dibubarkan (objek dihapus), maka secara alamiah list Prodi di dalamnya juga ikut musnah.

D. Class Main
- Bertindak sebagai 'Driver' / eksekutor untuk menjalankan simulasi

sesuai dengan skenario studi kasus.

- **Kode Program & Penjelasan**

Dosen.java

```
public class Dosen {
    private String nidn;
    private String nama;

    // Constructor
    public Dosen(String nidn, String nama) {
        this.nidn = nidn;
        this.nama = nama;
    }

    // Method mengajar
    public void mengajar() {
        System.out.println("Dosen " + this.nama + " (NIDN: " + this.nidn + ") sedang
mengajar.");
    }

    public String getNama() {
        return nama;
    }

    public void infoDosen() {
        System.out.println("- " + nama + " (NIDN: " + nidn + ")");
    }
}
```

Fakultas.java

```
import java.util.ArrayList;

public class Fakultas {
    private String namaFakultas;
    // Komposisi: Siklus hidup Prodi bergantung sepenuhnya pada Fakultas
    private ArrayList<Prodi> daftarProdi;
```

```
    // Constructor
    public Fakultas(String namaFakultas) {
        this.namaFakultas = namaFakultas;
        this.daftarProdi = new ArrayList<>();
    }

     // Method tambahProdi: Implementasi Komposisi (Objek diciptakan langsung melalui parent)
    public void tambahProdi(String namaProdi) {
         // Prodi di-instansiasi di dalam ruang kelas Fakultas. Jika Fakultas hancur, list Prodi-nya akan hancur
        Prodi prodiBaru = new Prodi(namaProdi);
        this.daftarProdi.add(prodiBaru);
         System.out.println("Berhasil menambahkan Prodi " + namaProdi + " ke Fakultas " + this.namaFakultas);
    }

     // Helper untuk mengambil prodi jika ingin difungsikan untuk manipulasi lebih lanjut (misal: tambah dosen)
    public Prodi getProdi(int index) {
        if(index >= 0 && index < daftarProdi.size()){
            return daftarProdi.get(index);
        }
        return null;
    }

    public void infoFakultas() {
        System.out.println("================================");
        System.out.println("Fakultas: " + namaFakultas);
        System.out.println("================================");
        for (Prodi p : daftarProdi) {
            p.infoProdi();
            System.out.println();
        }
    }
}
```

Main.Java

```
public class Main {
    public static void main(String[] args) {
        System.out.println("=== Inisialisasi Objek Independen (Agregasi) ===");
        // 1. Membuat Dosen. Disini ditunjukkan dosen hidup meski tanpa Prodi.
        Dosen dsn1 = new Dosen("1001", "Budi Santoso, M.Kom.");
        Dosen dsn2 = new Dosen("1002", "Siti Aminah, M.T.");
        Dosen dsn3 = new Dosen("1003", "Andi Hermawan, Ph.D.");

            System.out.println("Dosen berhasil dibuat tanpa perlu masuk ke Prodi mana
pun.\n");

        System.out.println("=== Inisialisasi Organisasi (Komposisi) ===");
        // 2. Membuat Fakultas dan langsung membuat Prodi di dalamnya.
        // Konsep Composition: instance Prodi tidak diinisialisasi terlebih dahulu,
        // melainkan terikat penciptaannya oleh ruang lingkup (instance) Fakultas-nya.
        Fakultas fasilkom = new Fakultas("Ilmu Komputer");
        fasilkom.tambahProdi("Teknik Informatika");
        fasilkom.tambahProdi("Sistem Informasi");

        System.out.println("\n=== Menghubungkan Dosen dan Prodi (Association) ===");
        // 3. Kita sambungkan Dosen yang sudah di daftar ke Prodi tersebut
        Prodi tek_info = fasilkom.getProdi(0); // Mendapatkan refrence Prodi TI
        Prodi sis_info = fasilkom.getProdi(1); // Mendapatkan refrence Prodi SI

            System.out.println("Menambahkan dosen ke Teknik Informatika dan Sistem
Informasi...");
        tek_info.tambahDosen(dsn1);
        tek_info.tambahDosen(dsn2);

        // Dosen bisa mengajar lebih dari 1 lokasi:
        sis_info.tambahDosen(dsn3);
        sis_info.tambahDosen(dsn1);

        System.out.println("\n=== MENAMPILKAN HASIL ===");
        // 4. Menampilkan Struktur Organisasi nya lengkap beserta Dosen
        fasilkom.infoFakultas();

        System.out.println("=== Bukti Aktifitas Individu Dosen ===");
        // Menampilkan Aktifitas Individu (Method mengajar())
```

```
        dsn1.mengajar();
        dsn2.mengajar();
        dsn3.mengajar();
    }
}
```

Prodi.Java

```
import java.util.ArrayList;

public class Prodi {
    private String namaProdi;
    // Agregasi: ArrayList menyimpan referensi objek Dosen yang berdiri sendiri
    private ArrayList<Dosen> daftarDosen;

    // Constructor
    public Prodi(String namaProdi) {
        this.namaProdi = namaProdi;
        this.daftarDosen = new ArrayList<>();
    }

    // Method tambahDosen (mengimplementasikan Agregasi dan Asosiasi)
    public void tambahDosen(Dosen dosen) {
        this.daftarDosen.add(dosen);
    }

    public String getNamaProdi() {
        return namaProdi;
    }

    public void infoProdi() {
        System.out.println("  Program Studi: " + namaProdi);
        System.out.println("  Daftar Dosen Pengajar:");
        if (daftarDosen.isEmpty()) {
            System.out.println("    (Belum ada dosen yang terdaftar)");
        } else {
            for (Dosen d : daftarDosen) {
                System.out.print("    ");
                d.infoDosen();
```

```
        }
    }
  }
}
```

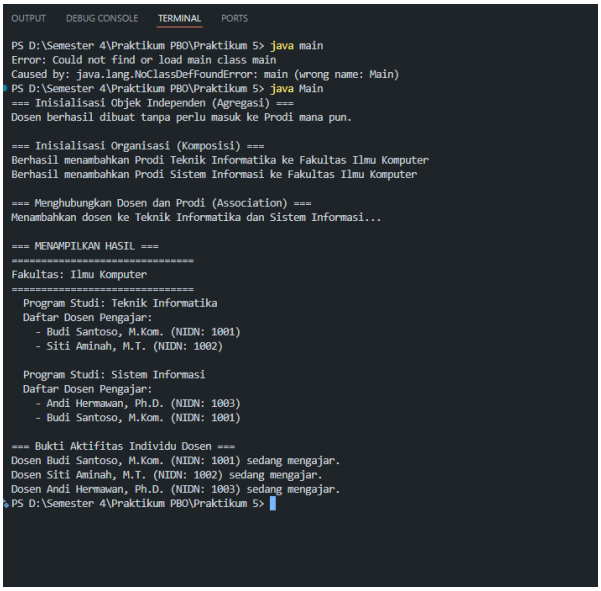
Penjelasan:

Program ini adalah simulasi sistem akademik sederhana menggunakan Java yang secara khusus di desain untuk mengimplementasikan konsep relasi antar objek (Class) dalam Object-Oriented Programming (OOP).

Fokus utama program ini adalah mendemonstrasikan 3 jenis relasi:

1. Agregasi: Objek Dosen diciptakan secara mandiri dan dapat berdiri sendiri tanpa harus terikat pada suatu Prodi tertentu.
2. Komposisi: Objek Prodi diciptakan oleh dan bergantung sepenuhnya pada objek Fakultas (siklus hidupnya terikat kuat).
3. Asosiasi: Objek Dosen dihubungkan kepada objek Prodi sebagai tenaga pengajar.

- **Hasil Running Program**



```
OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Semester 4\Praktikum PBO\Praktikum 5> java main
Error: Could not find or load main class main
Caused by: java.lang.NoClassDefFoundError: main (wrong name: Main)
PS D:\Semester 4\Praktikum PBO\Praktikum 5> java Main
=== Inisialisasi Objek Independen (Agregasi) ===
Dosen berhasil dibuat tanpa perlu masuk ke Prodi mana pun.

=== Inisialisasi Organisasi (Komposisi) ===
Berhasil menambahkan Prodi Teknik Informatika ke Fakultas Ilmu Komputer
Berhasil menambahkan Prodi Sistem Informasi ke Fakultas Ilmu Komputer

=== Menghubungkan Dosen dan Prodi (Association) ===
Menambahkan dosen ke Teknik Informatika dan Sistem Informasi...

=== MENAMPILKAN HASIL ===
==============================
Fakultas: Ilmu Komputer
==============================
  Program Studi: Teknik Informatika
  Daftar Dosen Pengajar:
    - Budi Santoso, M.Kom. (NIDN: 1001)
    - Siti Aminah, M.T. (NIDN: 1002)

  Program Studi: Sistem Informasi
  Daftar Dosen Pengajar:
    - Andi Hermawan, Ph.D. (NIDN: 1003)
    - Budi Santoso, M.Kom. (NIDN: 1001)

=== Bukti Aktifitas Individu Dosen ===
Dosen Budi Santoso, M.Kom. (NIDN: 1001) sedang mengajar.
Dosen Siti Aminah, M.T. (NIDN: 1002) sedang mengajar.
Dosen Andi Hermawan, Ph.D. (NIDN: 1003) sedang mengajar.
PS D:\Semester 4\Praktikum PBO\Praktikum 5>
```

- **Bagian A**

1. Menentukan Class yang Dibutuhkan

   Dari deskripsi soal, entitas utama yang menjadi Class adalah:

   - Fakultas
   - Prodi (Program Studi)
   - Dosen

2. Menentukan Atribut dan Method

   Class: Fakultas

   - Atribut:

    - namaFakultas (String)
    - daftarProdi (List/Array dari objek Prodi)
- Method:
    - tambahProdi(Prodi p): Untuk memasukkan prodi baru ke dalam fakultas.
    - tampilkanInfoFakultas(): Untuk mencetak nama fakultas beserta daftar prodi di dalamnya.

Class: Prodi

- Atribut:
    - namaProdi (String)
    - daftarDosen (List/Array dari objek Dosen)
- Method:
    - tambahDosen(Dosen d): Untuk mendaftarkan dosen ke prodi ini.
    - hapusDosen(Dosen d): Untuk menghapus/mengeluarkan dosen dari prodi.
    - tampilkanInfoProdi(): Untuk mencetak nama prodi beserta dosen yang mengajar di sana.

Class: Dosen

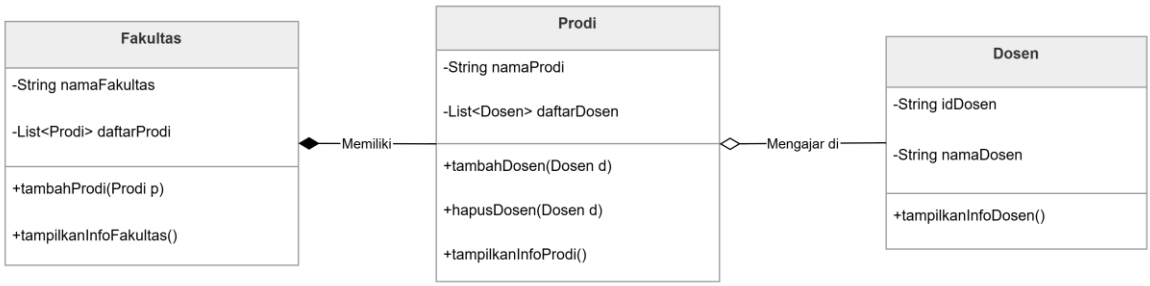
- Atribut:
    - idDosen (String)
    - namaDosen (String)
- Method:
    - tampilkanInfoDosen(): Untuk melihat detail data dosen.

3. Relasi Antar Class (UML)

   Fakultas ke Prodi ➔ Komposisi (Composition)

   - *Alasan:* Soal menyebutkan "Prodi berada di dalam Fakultas". Secara logika sistem OOP, sebuah Prodi tidak bisa berdiri sendiri jika Fakultasnya tidak ada. Jika Fakultas dihapus, maka Prodi di dalamnya ikut terhapus (siklus hidup bergantung erat).
   - *Simbol UML:* Garis dengan Belah ketupat hitam/penuh di sisi Fakultas.

   Prodi ke Dosen ➔ Agregasi (Aggregation)

   - *Alasan:* Soal secara eksplisit menyatakan "Dosen tetap ada walaupun tidak terdaftar di Prodi". Ini berarti Dosen dan Prodi memiliki siklus hidup yang independen. Jika sebuah Prodi ditutup/dihapus, data Dosen tersebut tidak ikut terhapus dari sistem.
   - *Simbol UML:* Garis dengan Belah ketupat putih/kosong di sisi Prodi.

Gambaran Visual Class Diagram: